[MP2] fileclose時obj所屬slab錯誤（已解決）

4 4

徐晨鈞 (HSU,CHEN-CHUN)

All Sections

想問大家，我在fileclose時obj所屬slab錯誤，但是一直找不出原因，想請教可能出問題的地方？

我有把struct run裡面新增一個struct slab* my_slab，指向obj所屬的slab

1. 進入fileclose時候一開始 **查看obj對應的slab是否正確** 的這個步驟發現問題
`debug("[fileclose] obj->my_slab: %p\n",((struct run*)f)->my_slab); /`
2. 我在fileclose的更動只有在最下方新增 `kmem_cache_free(file_cache, f);` 的操作
3. 我在kmem_cache_alloc最後return的是要分配出去的current_run

[FILE] filealloc
[SLAB] Alloc request on cache file_cache
[SLAB] A new slab 0x0000000087f4e000 (file_cache) is allocated
[SLAB] Object 0x0000000087f4e020 in slab 0x0000000087f4e000 (file_cache) is allocated ...

[FILE] filealloc
[SLAB] Alloc request on cache file_cache
[SLAB] Object **0x0000000087f4e218** in slab 0x0000000087f4e000 (file_cache) is allocated...

[FILE] fileclose
[fileclose] obj->my_slab: 0x0000000087f40101
[kmem_cache_free] Freeing obj: **0x0000000087f4e218**, my_slab: 0x0000000087f40101
[SLAB] Free **0x0000000087f4e218** in slab 0x0000000087f40101 (file_cache)
[SLAB] end of free

Search entries or author | Unread | Subscribe

Reply

楊子承 (YANG TZU CHENG) (https://cool.ntu.edu.tw/courses/46296/users/181911)

Sunday

我猜是因為你把 run 分配出去之後，那個記憶體有可能會被改掉，因此你存的 pointer 會失效。

有一個做法是把 obj/PGSIZE*PGSIZE 就會是 slab 的 pointer。

因為 kalloc 來的地址一定是 page size 的倍數的關係。



徐晨鈞 (HSU,CHEN-CHUN) (https://cool.ntu.edu.tw/courses/46296/users/191791)

Monday

有成功計算了！感謝

Reply



楊子慶 (YANG, TZU-CHING) (https://cool.ntu.edu.tw/courses/46296/users/229158)

Sunday

同學您好：

同學選擇在 obj 空間中存放 slab 地址的想法非常有趣，事實上也是 Linux kernel 比較古老時的做法，是比較有挑戰性的實作。這是因為當 obj 空間被配置出去後，obj 空間內的資料將會被解釋為特定的系統物件於核心中被使用，同學紀錄的 slab 地址將會受影響。

由於同學選擇的實作較有挑戰性，助教可以給同學概念上的提示：

首先原本在記憶體裡儲存鏈結序列指標 run->next 可行的其中一個前提是：我們並不在意該指標在配置出去以後的數值，因為其只負責維護 freelist 的狀態，被配置出去且 freelist 進行相應的更新後 run->next 便不會再被使用，而是在物件被釋放時重新指派為新的 run->next 值，也就是 run->next 數值的生命週期只需要在 slab 當中即可，離開 slab (被配置出去以後) 後結束其生命週期，這件事不影響後續運作的正確性。

不過同學今天的需求是希望在 obj 的記憶體中儲存 my_slab，其生命週期的要求是「從被配置起至被釋放」，與前述狀況不符，與物件被使用時的生命週期相重疊，故我們需要額外的記憶體進行 my_slab 的紀錄，亦即只預留 object_size 空間是不夠的。

若一個物件大小 504B，同學在配置時需要為每個 freelist 節點劃分 504 + 8 (一個指標大小) B 大小的空間 s，並在 s 中最前面 (推薦) 或最後面存放 my_slab，然後將物件所屬之 504B 空間的開頭返回。釋放時同學取得物件的地址後，要依照其地址反向找到儲存 my_slab 的地址，再進行後續的操作。

以分配時將 my_slab 放置於 s 最前面為例，配置出的物件地址應為 obj = s + 8 (一個指標大小)。進行釋放時取得 obj 後，要取得 my_slab 的存放位置只需要左右移向：addr(my_slab) =

s = obj - 8。

後日談：由於以上做法帶來的空間損失，以及利用分頁對齊特性的高效性，Linux kernel 在 slab 的繼任者 slub 系統中拋棄了這樣的做法。不過在本次作業中，助教鼓勵任何形式的做法，同學願意朝這個方向進行設計與實作很值得鼓勵，也會是很棒的系統程式訓練。

若有任何問題請不吝指教，感謝。

子慶



徐晨鈞 (HSU,CHEN-CHUN) (https://cool.ntu.edu.tw/courses/46296/users/191791)

Monday

了解了大概上述說明的實作方式，但是的確比其他方法複雜一些，但我還是選擇前一位同學回覆的方式，比較好實作

感謝助教用心的說明！

Reply